



中南大學
CENTRAL SOUTH UNIVERSITY

大规模集成电路实验报告

FIFO 功能验证与覆盖率分析

学生姓名：	樊博文
学 号：	8207230322
学 院：	自动化学院
专业班级：	电子信息工程 2303

本科生院制

2026 年 6 月

目录

一、实验目的	2
二、实验要求与完成情况	2
三、FIFO 被测模块原理分析	3
3.1 接口信号含义	3
3.2 写入条件	4
3.3 读取条件	4
3.4 空满标志判断	5
四、验证总体方案	5
五、测试平台设计	7
5.1 时钟与复位	7
5.2 激励施加时序	7
5.3 参考模型与 Scoreboard	8
5.4 测试场景设计	9
六、ModelSim 实验步骤	10
6.1 推荐 Transcript 命令	11
6.2 波形文件转换命令	11
七、覆盖率结果与分析	11
7.1 语句覆盖率分析	14
7.2 分支覆盖率分析	14
7.3 条件覆盖率未达到 100% 的原因	14
八、波形观察与结果说明	15
8.1 波形总览	15
8.2 复位与基本读写波形	16
8.3 写满与恢复波形	17
8.4 指针回绕波形	18
九、实验结论	18
附录 A: FIFO.v 源代码	20
附录 B: testbench.v 源代码	22
附录 C: 覆盖率报告关键摘录	26

一、实验目的

- 掌握同步 FIFO 被测模块的基本验证思路，理解读写请求、读写就绪、读写指针和空满标志之间的关系。
- 学习使用 Verilog 编写自校验 testbench，通过参考模型自动比较 DUT 输出结果，减少人工观察波形带来的遗漏。
- 练习定向激励与随机激励相结合的方法，覆盖复位、空读、基本写读、写满、满后恢复、并发读写、指针回绕和随机压力测试等场景。
- 掌握在 ModelSim 中启用代码覆盖率、保存 UCDB 数据库、导出覆盖率文本报告和分析未覆盖项的方法。
- 学习记录 VCD 波形并转换为 WLF 波形文件，能够从波形中解释关键时序现象。
- 理解覆盖率并不等价于功能完全正确：覆盖率是验证充分性的辅助指标，最终仍需结合测试目标、自动比较结果和波形进行综合判断。

二、实验要求与完成情况

学生自己练习从文件读取激励数据及结果数据、随机输入至测试模块的输入端口、自动比较输出结果、记录波形、利用覆盖率判断激励数据是否充分等验证方法。

结合本次 FIFO 验证对象，实验完成情况如下。

实验要求	本次实现方式	完成情况
随机输入至测试模块	在随机压力测试阶段连续执行 500 次随机读写请求，随机种子固定为 32'h13579bdf。	已完成
自动比较输出结果	testbench 内部建立参考 FIFO 模型，DUT 每次接受读操作时与模型队首数据比较。	已完成
记录波形	使用 \$dumpfile("sim.vcd") 与 \$dumpvars(0, testbench) 生成 VCD，并可转换为 sim.wlf。	已完成
覆盖率判断激励充分性	只对 FIFO.v 启用 Statement、Branch、Condition、Expression、FSM 覆盖率，导出 report_final.txt。	已完成
覆盖率未满足时分析原因	定位到 FIFO.v 第 75 行、第 85 行的特殊回绕判断，说明其属于冗余判断导致的条件覆盖率缺口。	已完成

三、FIFO 被测模块原理分析

本实验的被测对象为单时钟同步 FIFO。FIFO 的核心功能是先进先出：先写入的数据必须先被读出。模块中使用一个寄存器数组作为存储体，使用读指针 *rp* 指向当前待读数据，使用写指针 *wp* 指向下一次写入位置。复位信号 *RSTn* 为低有效异步复位。

3.1 接口信号含义

信号	方向	含义
<i>CLK</i>	input	时钟信号，写入、读指针更新和状态标志更新均在时钟上升沿进行。

$RSTn$	input	低有效异步复位信号，复位时读写指针清零，FIFO_wready 置 1，FIFO_rready 清 0。
$FIFO_w$	input	写请求信号。只有当 FIFO_w 为 1 且模块允许写入时，写数据才会被接受。
$FIFO_wdata$	input[31:0]	写入 FIFO 的数据。
$FIFO_wready$	output	写就绪信号，为 1 表示 FIFO 未滿，可以接受普通写入。
$FIFO_r$	input	读请求信号。只有当 FIFO_r 与 FIFO_rready 同时为 1 时，读操作有效。
$FIFO_rready$	output	读就绪信号，为 1 表示 FIFO 非空，可以读出数据。
$FIFO_rdata$	output[31:0]	读指针 rp 当前指向的数据，采用组合方式输出 FIFO[rp]。

3.2 写入条件

写入逻辑为：当 FIFO_w 为 1，并且满足 FIFO_r 为 1 或 FIFO_wready 为 1 时，数据写入 FIFO[wp]。也就是说，普通情况下 FIFO 未滿才允许写入；如果 FIFO 已滿但当前周期同时发生读取，则读取会释放一个位置，因此仍允许写入。这种设计可以支持滿状态下的并发读写，提高吞吐率。

```
if (FIFO_w && (FIFO_r || FIFO_wready))
    FIFO[wp] <= FIFO_wdata;
```

3.3 读取条件

FIFO_rdata 持续输出 FIFO[rp]，真正的读操作是否被接受由 FIFO_r &&

FIFO_rready 判断。当读操作被接受后，读指针 rp 在时钟上升沿加 1；当 rp 已为 4'hF 时，下一个位置回绕到 4'h0。

```
else if (FIFO_r && FIFO_rready)
    if (rp == {BITNUM{1'b1}})
        rp <= {BITNUM{1'b0}};
    else
        rp <= rp + 1'b1;
```

3.4 空满标志判断

FIFO_rready 表示 FIFO 是否非空。复位后 FIFO 为空，因此 FIFO_rready 为 0；当空 FIFO 中写入一个数据后，FIFO_rready 置 1；当读出最后一个数据且本周期没有同时写入时，FIFO_rready 清 0。

FIFO_wready 表示 FIFO 是否未滿。复位后 FIFO 可写，因此 FIFO_wready 为 1；当写入最后一个空闲位置且本周期没有同时读取时，FIFO_wready 清 0；当满 FIFO 中读出一个数据且本周期没有同时写入时，FIFO_wready 重新置 1。在本设计中，指针宽度为 4 位，深度为 16。当指针从 4'hF 加 1 时回绕到 4'h0；同时，4 位减法也会自动截断，例如 $4'h0 - 1'b1 = 4'hF$ 。这一特点会影响后续条件覆盖率分析。

四、验证总体方案

本实验采用定向边界测试 + 固定种子随机测试 + 参考模型自动比对的验证结构。定向测试保证关键边界场景一定出现，随机测试提高状态组合和读写序列的覆盖范围，参考模型负责在每次有效读操作时自动判断 FIFO 输出顺序是否正确。

在激励产生方面，测试平台通过 drive 任务向 DUT 施加写请求、读请求和写入数据。对于复位、空读、基本写读、写满、满后恢复、并发读写和指针回绕等关键场景，采用定向激励保证这些情况一定能够被测试到；在随机压力测试阶段，则通过固定随机种子产生 FIFO_w、FIFO_r 和 FIFO_wdata，使测试过程既具有随机性，又能够重复运行和复现问题。

在结果判断方面，测试平台内部建立了一个参考 FIFO 模型。该模型使用 model 数组保存期望数据，使用 model_head 指向当前应读出的队首数据，使用 model_tail 指向下一次写入位置，使用 model_count 记录当前模型中的有效数据个数。每当 DUT 接受一次有效写操作时，测试平台就把写入数据同步保存到参考模型中；每当 DUT 接受一次有效读操作时，测试平台就将 FIFO_rdata 与 model[model_head] 进行比较。如果二者不一致，说明 FIFO 的读出顺序或数据保存存在错误，测试平台会输出错误信息并增加 errors 计数。

在自动比较方面，本实验并不是只通过观察波形来判断 FIFO 是否正确，而是在 testbench 中加入了 scoreboard 机制。scoreboard 会根据 DUT 的握手条件判断一次读写操作是否真正被接受，并在读操作有效时自动比较实际输出和期望输出。这样可以及时发现数据顺序错误、空 FIFO 被错误读取、满 FIFO 被错误写入等问题，提高验证的可靠性。

在覆盖率统计方面，本实验只对被测文件 FIFO.v 启用 Statement、Branch、Condition、Expression 和 FSM 覆盖率，用于判断测试激励是否充分触发了 FIFO 设计中的主要代码路径和条件表达式。通过覆盖率报告，可以进一步分析哪些语句、分支或条件没有被覆盖，并判断原因是激励不足还是代码中存在冗余逻辑。

在波形观察方面，测试平台同时记录仿真波形，用于辅助分析 FIFO 的时序

行为。重点观察复位后读写指针和空满标志的状态、基本写读过程中的数据传递、FIFO 写满后的 FIFO_wready 变化、满状态读取后的恢复过程、读写指针从 4'hF 回绕到 4'h0 的过程，以及随机测试阶段的整体读写活动。通过自动比较、覆盖率统计和波形观察三者结合，可以较全面地验证 FIFO 模块的功能正确性。

五、测试平台设计

5.1 时钟与复位

测试平台中时钟周期 PERIOD 设置为 10 ns，初值为 0，每 5 ns 翻转一次。

复位任务 apply_reset 在时钟下降沿拉低 RSTn，保持 3 个下降沿后释放复位，并检查复位后的空满标志是否符合预期。

```
parameter PERIOD = 10;
initial begin
    CLK = 1'b0;
    forever #(PERIOD/2) CLK = ~CLK;
end
```

5.2 激励施加时序

drive 任务在时钟下降沿改变输入信号，在下一个下降沿撤销请求。这样 DUT 在上升沿采样时，输入信号已经稳定，能够避免激励变化与 DUT 采样发生在同一仿真时刻造成竞争。

```
task drive;
    input do_write;
    input do_read;
```

```
input [WIDTH-1:0] data;
begin
    @(negedge CLK);
    FIFO_w      = do_write;
    FIFO_r      = do_read;
    FIFO_wdata = data;
    @(negedge CLK);
    FIFO_w      = 1'b0;
    FIFO_r      = 1'b0;
end
endtask
```

5.3 参考模型与 Scoreboard

测试平台中的参考模型等价于一个理想 FIFO。model_head 指向期望读出的队首数据, model_tail 指向下一次写入位置, model_count 记录参考 FIFO 中的有效数据个数。每当 DUT 接受一次读操作, scoreboard 就将 FIFO_rdata 与 model[model_head] 比较; 每当 DUT 接受一次写操作, scoreboard 将 FIFO_wdata 写入参考模型。

这里特别注意一个时序细节: DUT 的读指针 rp 在时钟上升沿使用非阻塞赋值更新, 而 FIFO_rdata 是由当前 rp 组合输出。因此在同一个上升沿的活动事件区中, scoreboard 看到的是旧读指针对应的数据, 正好应与 model_head 当前数据比较。这也是注释中提到 “samples the old read pointer” 的原因。

```
if (FIFO_r && FIFO_rready) begin
    if (FIFO_rdata != model[model_head]) begin
        $display("ERROR @%0t: expected %h, got %h", $time,
model[model_head], FIFO_rdata);
        errors = errors + 1;
    end
    model_head = (model_head + 1) % DEPTH;
    model_count = model_count - 1;
end
```

5.4 测试场景设计

测试阶段	激励内容	验证目标
复位检查	拉低 RSTn 后释放复位。	检查 rp、wp 清零, FIFO_wready=1, FIFO_rready=0。
空读拒绝	FIFO 为空时施加 FIFO_r。	确认空 FIFO 不应推进读指针, 也不应产生有效数据比较。
基本写读	写入 32'h12345678 后读取。	确认写入后 FIFO_rready 拉高, 读出数据顺序正确。
写满测试	连续写入 16 项数据。	确认 FIFO 达到满状态后 FIFO_wready 拉低。
满写拒绝	满状态下继续写入 32'hDEADBEEF。	确认非法写入不被接受, 写指针不错误前进。
满后恢复	满状态下读出一个数据。	确认释放一个位置后 FIFO_wready 恢复为 1。
并发读写	连续 40 次 FIFO_w 与 FIFO_r 同时有效。	验证满吞吐路径、读写同时发生和指针环绕。
读空检查	持续读取直到 FIFO_rready 清 0。	确认读出最后一个数据后 FIFO 进入空状态。
随机压力	500 次固定种子随机读写。	提高组合状态覆盖率, 检查长序列下数据顺序仍正确。
最终排空	随机阶段后继续读取到空。	保证剩余数据全部经过自动比较。

六、ModelSim 实验步骤

以下步骤为本实验实际使用的操作流程。为避免 testbench 内部错误处理分支影响被测对象覆盖率统计,覆盖率只对 FIFO.v 启用,不对 testbench.v 启用。

1. 在 test10/ 目录建立工程, 将 FIFO.v 与 testbench.v 加入工程。
2. 在 FIFO.v 的 Properties - Coverage 中勾选 Statement、Branch、Condition、Expression 和 Finite State Machine Coverage。

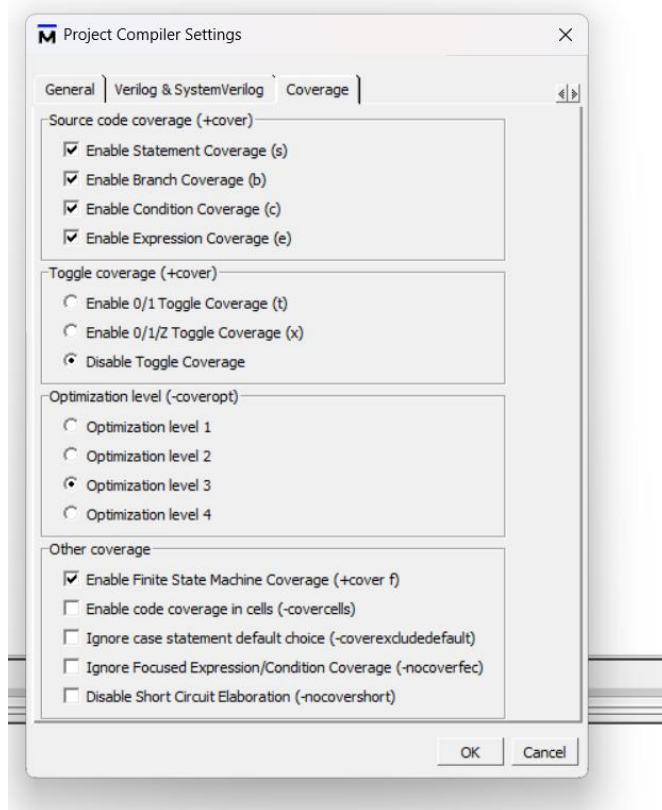


图 1 FIFO.v 覆盖率选项设置

3. 编译 FIFO.v, 并只对 FIFO.v 启用覆盖率; 再单独编译 testbench.v。
4. 以 work.testbench 为顶层进入仿真, 并启用 coverage。
5. 添加需要观察的信号到 Wave 窗口, 执行 run -all。
6. 仿真结束后保存 test10_final.ucdb, 并导出 report_final.txt。
7. 转换 sim.vcd 为 sim.wlf, 或直接打开 vsim.wlf 查看波形。

6.1 推荐 Transcript 命令

```
vlib work
vlog -cover bcesf FIFO.v
vlog testbench.v
vsim -coverage work.testbench
add wave -r sim:/testbench/*
run -all
coverage save test10_final.ucdb
coverage report -file report_final.txt -details -all
```

其中 -cover bcesf 的含义为：b 表示 Branch Coverage, c 表示 Condition Coverage, e 表示 Expression Coverage, s 表示 Statement Coverage, f 表示 FSM Coverage。

6.2 波形文件转换命令

testbench 会生成 sim.vcd。若需要转换为 WLF, 应先退出当前仿真, 保证 VCD 文件写入结束标记。

```
quit -sim
exec vcd2wlf sim.vcd sim.wlf
vsim -view sim.wlf
add wave -r /*
wave zoom full
```

七、覆盖率结果与分析

FIFO.v 与 testbench.v 编译成功后运行仿真, 测试平台完成全部定向测试与 500 次随机压力测试。仿真末尾输出 TEST PASS: all FIFO comparisons passed, 说明整个测试过程中参考模型未发现数据顺序错误, 也未发现 DUT 接受非法读写的情况。

```

Transcript
# Reading D:/Modelsim/tcl/vsim/pref.tcl
# // ModelSim SE-64 10.7 Dec 7 2017
# //
# // Copyright 1991-2017 Mentor Graphics Corporation
# // All Rights Reserved.
# //
# // ModelSim SE-64 and its associated documentation contain trade
# // secrets and commercial or financial information that are the property of
# // Mentor Graphics Corporation and are privileged, confidential,
# // and exempt from disclosure under the Freedom of Information Act,
# // 5 U.S.C. Section 552. Furthermore, this information
# // is prohibited from disclosure under the Trade Secrets Act,
# // 18 U.S.C. Section 1905.
# //
# Project file D:/Users/86187/Desktop/LSI_experiments/test10/test10.mpf was not found
# Unable to open project.
# Loading project test10
# Compile of FIFO.v was successful.
# Compile of testbench.v was successful.
# 2 compiles, 0 failed with no errors.
ModelSim>vlog -cover bcsf FIFO.v
# Model Technology ModelSim SE-64 vlog 10.7 Compiler 2017.12 Dec 7 2017
# Start time: 19:54:08 on Jun 22,2026
# vlog -reportprogress 300 -cover bcsf FIFO.v
# -- Compiling module FIFO
#
# Top level modules:
#     FIFO
# End time: 19:54:08 on Jun 22,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
ModelSim>vlog testbench.v
# Model Technology ModelSim SE-64 vlog 10.7 Compiler 2017.12 Dec 7 2017
# Start time: 19:54:08 on Jun 22,2026
# vlog -reportprogress 300 testbench.v
# -- Compiling module testbench
#
# Top level modules:
#     testbench
# End time: 19:54:08 on Jun 22,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
ModelSim>vsim -coverage work.testbench
# vsim -coverage work.testbench
# Start time: 19:54:17 on Jun 22,2026
# ** Note: (vsim-3812) Design is being optimized...
# Loading work.testbench(fast)
# Loading work.FIFO(fast)
VSIM 4>add wave -r sim:/testbench/*
VSIM 5>run -all
# TEST PASS: all FIFO comparisons passed
# ** Note: $finish      : testbench.v(174)
#     Time: 11775 ns  Iteration: 1  Instance: /testbench
# 1
# Break at testbench.v line 174
VSIM 6>coverage save test10_final.ucdb
VSIM 7>coverage report -file report_final.txt -details -all
VSIM 8>

```

图 2 编译、仿真与自动比较通过结果

该图显示 FIFO.v 和 testbench.v 均编译成功, Errors 和 Warnings 均为 0; 执行 run -all 后输出 TEST PASS: all FIFO comparisons passed, 说明参考模型自动比较未发现错误

本实验导出的覆盖率报告为 report_final.txt。由于覆盖率只针对 FIFO.v 统计，因此此结果能够反映被测 FIFO 设计本身的执行充分性，而不会被 testbench 中错误处理分支拉低。

覆盖率类型	有效项	已覆盖	未覆盖	覆盖率
Statement	19	19	0	100.00%
Branch	20	20	0	100.00%
Condition	25	21	4	84.00%
Total	-	-	-	94.66%

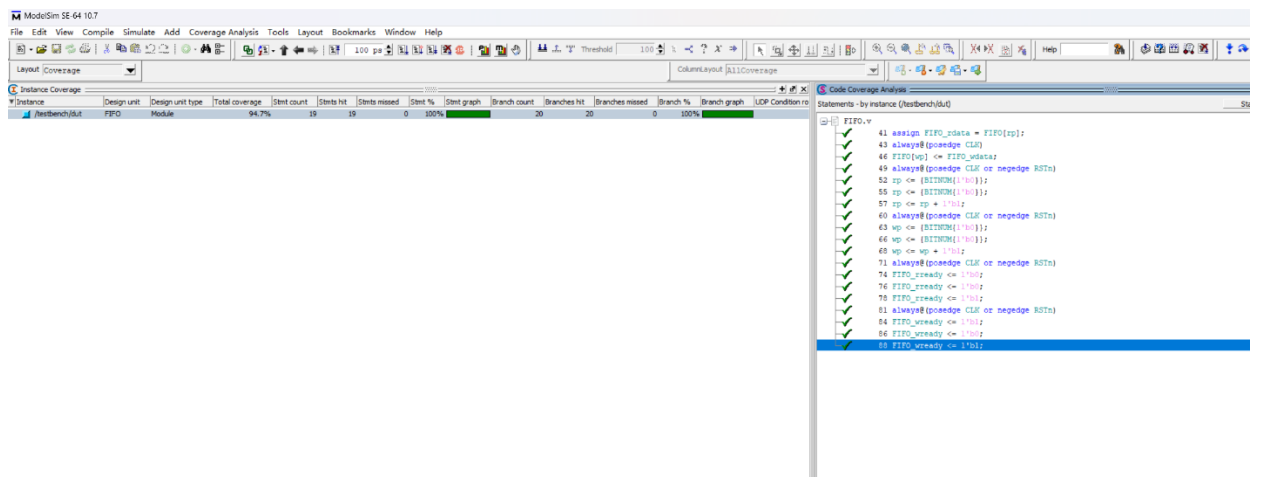


图 3 FIFO 语句覆盖率和分支覆盖率结果

该图显示 Statement 覆盖率为 100%，Branch 覆盖率为 100%，说明 FIFO.v 中所有可执行语句和主要分支均已被测试激励触发。

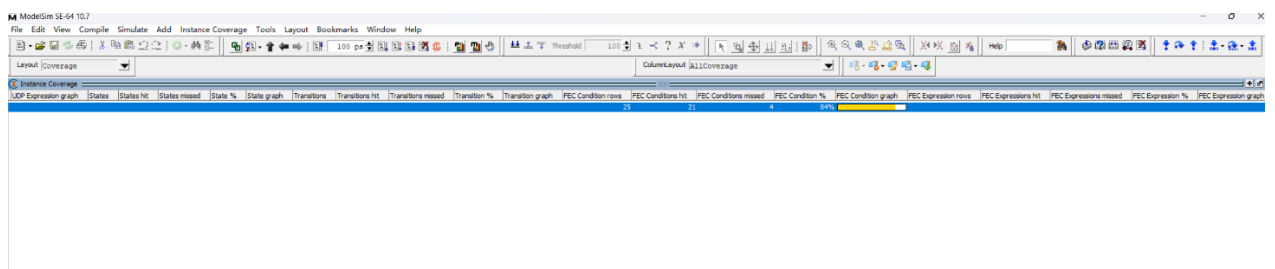


图 4 FIFO 条件覆盖率结果

该图显示 FEC Condition rows 为 25，Conditions hit 为 21，Conditions

missed 为 4, Condition 覆盖率为 84%。这与报告中分析的第 75 行和第 85 行特殊回绕条件未完全覆盖相对应。

7.1 语句覆盖率分析

语句覆盖率为 100%，说明 FIFO.v 中所有可执行语句均至少执行过一次。复位语句、读指针加 1 与回绕、写指针加 1 与回绕、读就绪置位/清零、写就绪置位/清零、存储器写入语句都被测试激励触发。

7.2 分支覆盖率分析

分支覆盖率为 100%，说明主要 if/else 分支的真分支、假分支或 all false 分支均被执行。该结果表明测试用例不仅覆盖了普通读写，还覆盖了空读、写满、满后恢复、指针回绕等边界场景。

7.3 条件覆盖率未达到 100% 的原因

条件覆盖率为 84%，共 25 个条件项中覆盖 21 个，未覆盖 4 个。未覆盖项位于 FIFO.v 第 75 行和第 85 行，分别是读空判断和写满判断中的特殊回绕判断。

Line 75:

```
((rp == wp - 1'b1) || ((rp == {BITNUM{1'b1}}) && (wp == {BITNUM{1'b0}}))))
```

Line 85:

```
((wp == rp - 1'b1) || ((wp == {BITNUM{1'b1}}) && (rp == {BITNUM{1'b0}}))))
```

以第 75 行为例,当 $wp=4'h0$ 时,4 位减法 $wp - 1'b1$ 的结果会自动截断为 $4'hF$ 。因此条件 $rp == wp - 1'b1$ 已经能够表达 $rp=4'hF$ 且 $wp=4'h0$ 的回绕关系。后半部分 $((rp == 4'hF) \&\& (wp == 4'h0))$ 与前半部分在这一状态下等价,不能独立改变整个 OR 表达式的结果。ModelSim 的 FEC 条件覆盖要求某个输入项能够在非遮蔽条件下独立影响表达式结果,因此该冗余项无法被单独覆盖。

第 85 行同理：当 $rp=4'h0$ 时， $rp - 1'b1$ 等于 $4'hF$ ，因此 $wp == rp - 1'b1$ 已经覆盖 $wp=4'hF$ 且 $rp=4'h0$ 的写满回绕关系。后半部分 $((wp == 4'hF) \&\& (rp == 4'h0))$ 也是冗余判断。

因此，本实验虽然条件覆盖率不是 100%，但未覆盖项属于冗余逻辑导致的不可独立覆盖项，不是测试激励不足。结合实验要求“覆盖率尽可能达到 100%，冗余设计除外”，该覆盖率结果可以接受。

八、波形观察与结果说明

波形分析的目标不是重复自动比较的功能，而是从时序角度确认复位、读写握手、空满标志变化和指针回绕是否符合 FIFO 设计预期。建议在 Wave 窗口至少观察 CLK、RSTn、FIFO_w、FIFO_wready、FIFO_wdata、FIFO_r、FIFO_rready、FIFO_rdata、dut/rp、dut/wp。数据与指针信号建议设置为十六进制显示。

8.1 波形总览

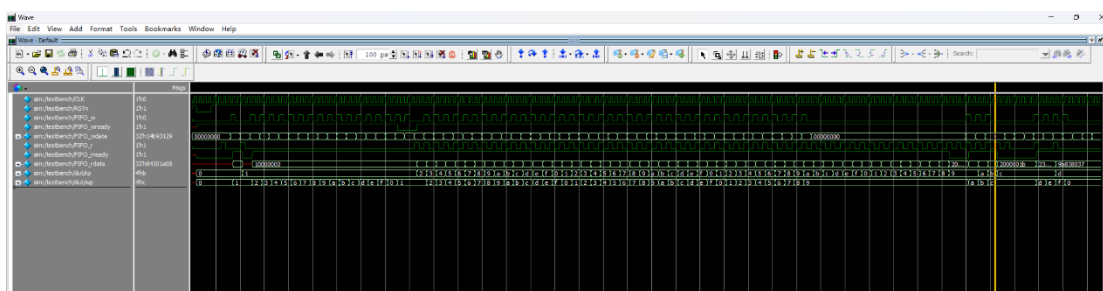


图 5 FIFO 仿真波形总览

波形总览中可以看到：前半段为有明确目的的定向测试，读写请求变化规律较清晰；后半段为随机压力测试，FIFO_w、FIFO_r 和 FIFO_wdata 呈随机变化。由于 scoreboard 在仿真过程中持续工作，只要任意一次数据顺序错误都会输出 ERROR 并

增加 errors 计数, 因此总览波形与 TEST PASS 共同说明长序列操作未破坏 FIFO 顺序。

8.2 复位与基本读写波形

wave zoom range 0ns 120ns

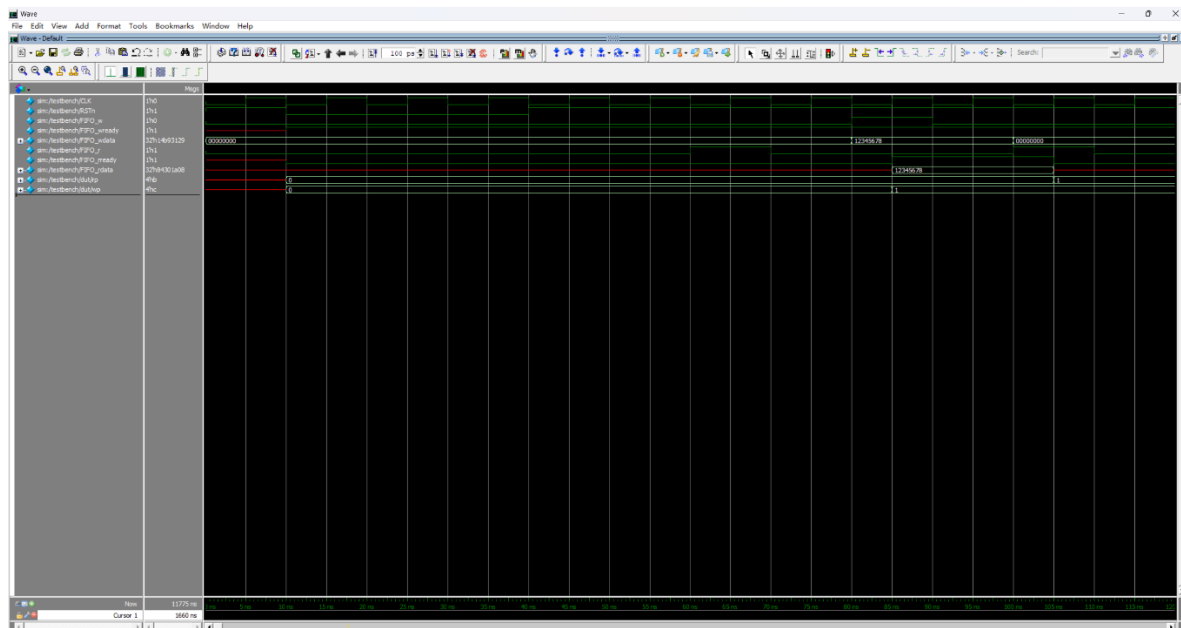


图 6 复位与基本读写波形

该图对应 0-120 ns 附近的波形。该时间段应能观察到：RSTn 拉低时，rp 与 wp 被清零；复位释放后，FIFO_wready 为 1、FIFO_rready 为 0；空读请求不会使 rp 前进；写入 32'h12345678 后 FIFO_rready 变为 1；随后读取时 FIFO_rdata 输出 32'h12345678，读指针前进。该现象证明基本写入、读取和空状态控制正确。

8.3 写满与恢复波形

wave zoom range 380ns 480ns

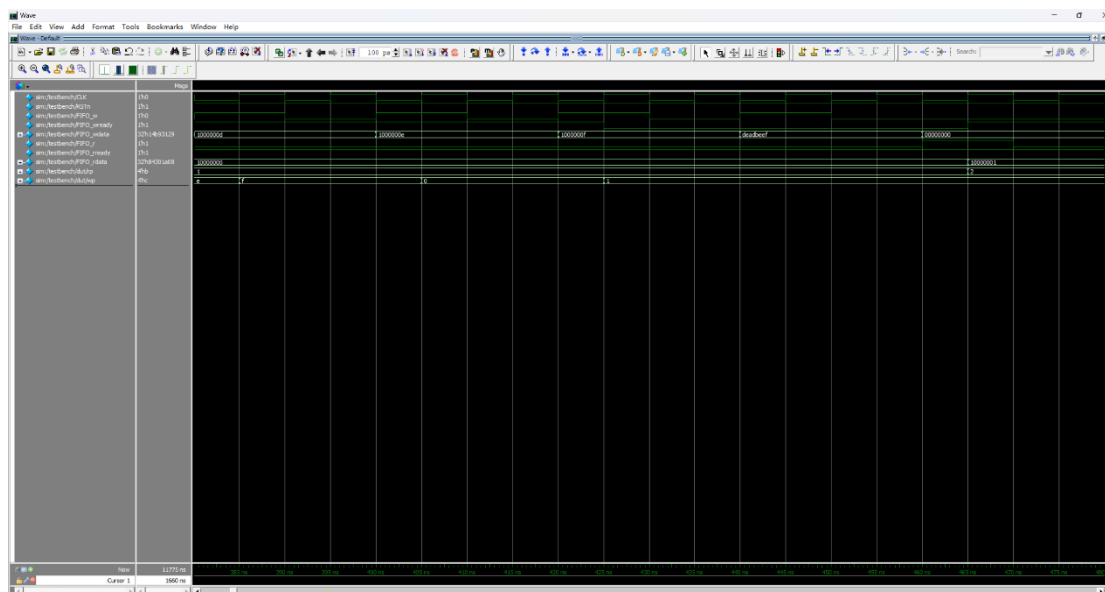


图 7 FIFO 写满、满写拒绝与读取恢复波形

该时间段应能观察到：连续写入第 16 个数据后，FIFO_wready 拉低，表示 FIFO 已满；满状态下继续施加写入 32'hDEADBEEF 时，写指针不应继续前进，说明额外写入未被接受；随后执行一次读取后释放一个存储位置，FIFO_wready 恢复为 1。该现象验证了满状态判断和满后恢复逻辑。

8.4 指针回绕波形

wave zoom range 730ns 810ns

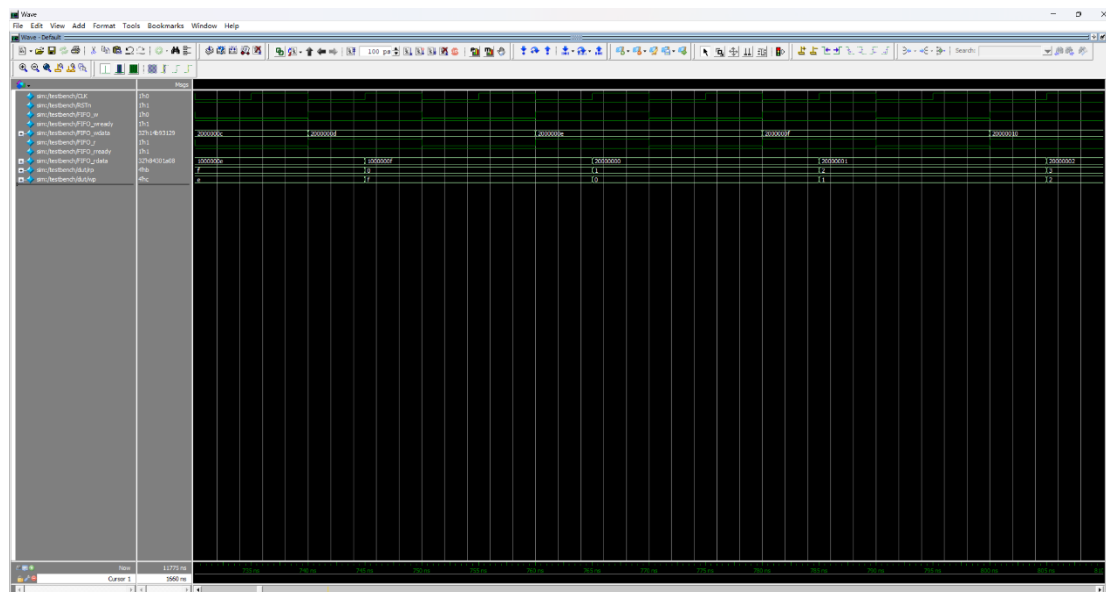


图 8 读写指针回绕波形

在并发读写阶段，FIFO_w 与 FIFO_r 同时有效，读写指针均持续前进。当指针值到达 4'hF 后，下一个有效操作会使其回绕到 4'h0。若回绕过程中 FIFO_rdata 仍按参考模型顺序输出，说明环形缓冲区地址更新正确。

九、实验结论

本实验完成了 16 x 32 bit 同步 FIFO 的功能验证与覆盖率分析。测试平台通过定向边界场景覆盖复位、空读、基本写读、写满、满写拒绝、满后恢复、并发读写、读空和指针回绕；通过 500 次固定种子随机读写进一步提高状态组合覆盖；通过参考 FIFO 模型在每一次有效读操作时自动比较输出数据，最终仿真结果为 TEST PASS，说明 DUT 在本实验测试范围内满足先进先出数据顺序要求。

覆盖率方面，FIFO.v 的语句覆盖率为 100%，分支覆盖率为 100%，条件覆盖率为 84%，综合覆盖率为 94.66%。4 个未覆盖条件项均来自第 75 行和第 85 行的特殊

回绕判断。由于 4 位指针减法本身已经具有回绕效果, 这些特殊判断无法独立影响 OR 表达式结果, 属于冗余设计导致的覆盖率缺口, 不属于测试激励不足。

通过本实验可以看出, 一个较完整的验证流程应同时包含测试目标设计、自动比较、波形记录、覆盖率统计和未覆盖项解释。覆盖率能够提示哪些代码路径或条件项没有被充分触发, 但是否需要继续补充激励, 还必须结合设计逻辑本身进行判断。

附录 A: FIFO.v 源代码

```
// FIFO(First Input First Output)
// Function:
//      Write operation is allowed only when FIFO_wready is '1'. To write data to FIFO,
//      FIFO_w and FIFO_wdata should be asserted.
//      Read operation is allowed only when FIFO_rready is '1'. To read data from FIFO,
//      FIFO_r should be asserted.

module FIFO(
    CLK,
    RSTn,
    FIFO_wready,
    FIFO_w,
    FIFO_wdata,
    FIFO_rready,
    FIFO_r,
    FIFO_rdata
);

    parameter FIFOWIDTH = 32;
    parameter FIFODEPTH = 16;
    parameter BITNUM = 4;

    input      CLK;
    input      RSTn;
    output     FIFO_wready;
    input      FIFO_w;
    input [(FIFOWIDTH-1):0] FIFO_wdata;
    output     FIFO_rready;
    input      FIFO_r;
    output [(FIFOWIDTH-1):0] FIFO_rdata;

    reg [(FIFOWIDTH-1):0] FIFO[0:(FIFODEPTH-1)];
    reg [(BITNUM-1):0]    rp;
    reg [(BITNUM-1):0]    wp;
    reg    FIFO_wready;
    reg    FIFO_rready;

    assign FIFO_rdata = FIFO[rp];
```

```

always@(posedge CLK)
begin
    if (FIFO_w && (FIFO_r || FIFO_wready))
        FIFO[wp] <= FIFO_wdata;
end

always@(posedge CLK or negedge RSTn)
begin
    if (!RSTn)
        rp <= {BITNUM{1'b0}};
    else if (FIFO_r && FIFO_rready)
        if (rp == {BITNUM{1'b1}})
            rp <= {BITNUM{1'b0}};
        else
            rp <= rp + 1'b1;
end

always@(posedge CLK or negedge RSTn)
begin
    if (!RSTn)
        wp <= {BITNUM{1'b0}};
    else if (FIFO_w && (FIFO_r || FIFO_wready))
        if (wp == {BITNUM{1'b1}})
            wp <= {BITNUM{1'b0}};
        else
            wp <= wp + 1'b1;
end

always@(posedge CLK or negedge RSTn)
begin
    if (!RSTn)
        FIFO_rready <= 1'b0;
    else if (((rp == wp - 1'b1) || ((rp == {BITNUM{1'b1}}) && (wp == {BITNUM{1'b0}})))) && FIFO_r && (!FIFO_w))
        FIFO_rready <= 1'b0;
    else if (!FIFO_rready && FIFO_w)
        FIFO_rready <= 1'b1;
end

always@(posedge CLK or negedge RSTn)
begin
    if (!RSTn)
        FIFO_wready <= 1'b1;

```

```

        else if (((wp == rp - 1'b1) || ((wp == {BITNUM{1'b1}}) && (rp ==
{BITNUM{1'b0}}))) && FIFO_w && (!FIFO_r))
            FIFO_wready <= 1'b0;
        else if (!FIFO_wready && FIFO_r && !FIFO_w)
            FIFO_wready <= 1'b1;
    end

endmodule

```

附录 B: testbench.v 源代码

```

`timescale 1ns/1ps

module testbench;
    parameter PERIOD = 10;
    parameter WIDTH  = 32;
    parameter DEPTH  = 16;

    reg          CLK;
    reg          RSTn;
    reg          FIFO_w;
    reg [WIDTH-1:0] FIFO_wdata;
    reg          FIFO_r;
    wire         FIFO_wready;
    wire         FIFO_rready;
    wire [WIDTH-1:0] FIFO_rdata;

    reg [WIDTH-1:0] model [0:DEPTH-1];
    integer model_head;
    integer model_tail;
    integer model_count;
    integer errors;
    integer i;
    integer seed;

    FIFO #(
        .FIFOWIDTH(WIDTH),
        .FIFODEPTH(DEPTH),
        .BITNUM(4)
    ) dut (
        .CLK(CLK),
        .RSTn(RSTn),
        .FIFO_wready(FIFO_wready),

```

```

.FIFO_w(FIFO_w),
.FIFO_wdata(FIFO_wdata),
.FIFO_rready(FIFO_rready),
.FIFO_r(FIFO_r),
.FIFO_rdata(FIFO_rdata)
);

initial begin
    CLK = 1'b0;
    forever #(PERIOD/2) CLK = ~CLK;
end

// The scoreboard samples the old read pointer in the active event region.
always @(posedge CLK) begin
    if (!RSTn) begin
        model_head = 0;
        model_tail = 0;
        model_count = 0;
    end else begin
        if (FIFO_r && FIFO_rready) begin
            if (model_count <= 0) begin
                $display("ERROR: DUT accepted a read while model FIFO was
empty");

                errors = errors + 1;
            end else begin
                if (FIFO_rdata !== model[model_head]) begin
                    $display("ERROR @%0t: expected %h, got %h",
                        $time, model[model_head], FIFO_rdata);
                    errors = errors + 1;
                end
                model_head = (model_head + 1) % DEPTH;
                model_count = model_count - 1;
            end
        end
    end

    if (FIFO_w && (FIFO_r || FIFO_wready)) begin
        if (model_count >= DEPTH) begin
            // A simultaneous accepted read creates one free entry.
            if (!(FIFO_r && FIFO_rready)) begin
                $display("ERROR: DUT accepted a write while model FIFO was
full");

                errors = errors + 1;
            end
        end
    end
end

```

```

        model[model_tail] = FIFO_wdata;
        model_tail = (model_tail + 1) % DEPTH;
        model_count = model_count + 1;
    end
end

task drive;
    input do_write;
    input do_read;
    input [WIDTH-1:0] data;
    begin
        @(negedge CLK);
        FIFO_w      = do_write;
        FIFO_r      = do_read;
        FIFO_wdata = data;
        @(negedge CLK);
        FIFO_w      = 1'b0;
        FIFO_r      = 1'b0;
    end
endtask

task apply_reset;
    begin
        @(negedge CLK);
        RSTn      = 1'b0;
        FIFO_w     = 1'b0;
        FIFO_r     = 1'b0;
        FIFO_wdata = {WIDTH{1'b0}};
        repeat (3) @(negedge CLK);
        RSTn = 1'b1;
        @(negedge CLK);
        if (FIFO_wready !== 1'b1 || FIFO_rready !== 1'b0) begin
            $display("ERROR: ready flags are wrong after reset");
            errors = errors + 1;
        end
    end
endtask

initial begin
    errors = 0;
    seed = 32'h13579bdf;
    RSTn = 1'b1;
    FIFO_w = 1'b0;

```

```
FIFO_r = 1'b0;
FIFO_wdata = 0;

$dumpfile("sim.vcd");
$dumpvars(0, testbench);

apply_reset;

// Empty-read rejection and basic write/read.
drive(1'b0, 1'b1, 32'h0);
drive(1'b1, 1'b0, 32'h12345678);
drive(1'b0, 1'b1, 32'h0);

// Fill all 16 entries, reject a 17th write, then recover with a read.
for (i = 0; i < DEPTH; i = i + 1)
    drive(1'b1, 1'b0, 32'h10000000 + i);
if (FIFO_wready !== 1'b0) begin
    $display("ERROR: FIFO did not report full");
    errors = errors + 1;
end
drive(1'b1, 1'b0, 32'hdeadbeef);
drive(1'b0, 1'b1, 32'h0);
if (FIFO_wready !== 1'b1) begin
    $display("ERROR: FIFO did not recover after reading from full state");
    errors = errors + 1;
end

// Simultaneous read/write exercises the full-throughput path and wraparound.
for (i = 0; i < 40; i = i + 1)
    drive(1'b1, 1'b1, 32'h20000000 + i);

// Drain to empty and verify the empty indication.
while (FIFO_rready)
    drive(1'b0, 1'b1, 32'h0);
drive(1'b0, 1'b1, 32'h0);

// Constrained random stress. The model accepts only legal DUT transfers.
for (i = 0; i < 500; i = i + 1) begin
    @(negedge CLK);
    FIFO_w = $random(seed) & 1'b1;
    FIFO_r = $random(seed) & 1'b1;
    FIFO_wdata = $random(seed);
    @(negedge CLK);
    FIFO_w = 1'b0;
```

```

        FIFO_r = 1'b0;
    end

    while (FIFO_rready)
        drive(1'b0, 1'b1, 32'h0);

    repeat (3) @(posedge CLK);
    if (errors == 0)
        $display("TEST PASS: all FIFO comparisons passed");
    else
        $display("TEST FAIL: %0d error(s)", errors);
    $finish;
    end
endmodule

```

附录 C：覆盖率报告关键摘录

Coverage Report by file with details

File: FIFO.v

Statement Coverage:

Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	-----	-----	-----
Stmts	19	19	0	100.00

Statement Details

Statement Coverage for file FIFO.v --

1

// FIFO(First Input First Output)

2

// Function:

3

// Write operation is allowed only when FIFO_wready is '1'.

To write data to FIFO, FIFO_w and FIFO_wdata should be asserted.

4

// Read operation is allowed only when FIFO_rready is '1'. To

read data from FIFO, FIFO_r should be asserted.

5

6

module FIFO(

7

CLK,

8

RSTn,

9

FIFO_wready,

10

FIFO_w,

```

11         FIFO_wdata,
12         FIFO_rready,
13         FIFO_r,
14         FIFO_rdata
15     );
16
17     parameter FIFOWIDTH = 32;
18     parameter FIFODEPTH = 16;
19     parameter BITNUM = 4;
20
21     input      CLK;
22     input      RSTn;
23     output     FIFO_wready;
24     input      FIFO_w;
25     input [(FIFOWIDTH-1):0]    FIFO_wdata;
26     output     FIFO_rready;
27     input      FIFO_r;
28     output [(FIFOWIDTH-1):0]    FIFO_rdata;
29
30
31
32
33     reg [(FIFOWIDTH-1):0]    FIFO[0:(FIFODEPTH-1)];
34     reg [(BITNUM-1):0]      rp;
35     reg [(BITNUM-1):0]      wp;
36     reg      FIFO_wready;
37     reg      FIFO_rready;
38
39
40
41         1          427    assign FIFO_rdata = FIFO[rp];
42
43         1          1032   always@(posedge CLK)
44             begin
45                 if (FIFO_w && (FIFO_r || FIFO_wready))
46                     1          288      FIFO[wp] <= FIFO_wdata;
47             end
48
49         1          611    always@(posedge CLK or negedge RSTn)
50             begin
51                 if (!RSTn)
52                     1          2          rp <= {BITNUM{1'b0}};
53                 else if (FIFO_r && FIFO_rready)
54                     if (rp == {BITNUM{1'b1}})

```

55	1	18	rp <= {BITNUM{1'b0}};
56			else
57	1	270	rp <= rp + 1'b1;
58			end
59			
60	1	894	always@(posedge CLK or negedge RSTn)
61			begin
62			if (!RSTn)
63	1	2	wp <= {BITNUM{1'b0}};
64			else if (FIFO_w && (FIFO_r FIFO_wready))
65			if (wp == {BITNUM{1'b1}})
66	1	18	wp <= {BITNUM{1'b0}};
67			else
68	1	270	wp <= wp + 1'b1;
69			end
70			
71	1	894	always@(posedge CLK or negedge RSTn)
72			begin
73			if (!RSTn)
74	1	2	FIFO_rready <= 1'b0;
75			else if (((rp == wp - 1'b1) ((rp == {BITNUM{1'b1}}) &&
			(wp == {BITNUM{1'b0}}))) && FIFO_r && (!FIFO_w))
76	1	14	FIFO_rready <= 1'b0;
77			else if (!FIFO_rready && FIFO_w)
78	1	14	FIFO_rready <= 1'b1;
79			end
80			
81	1	894	always@(posedge CLK or negedge RSTn)
82			begin
83			if (!RSTn)
84	1	2	FIFO_wready <= 1'b1;
85			else if (((wp == rp - 1'b1) ((wp == {BITNUM{1'b1}}) &&
			(rp == {BITNUM{1'b0}}))) && FIFO_w && (!FIFO_r))
86	1	20	FIFO_wready <= 1'b0;
87			else if (!FIFO_wready && FIFO_r && !FIFO_w)
88	1	20	FIFO_wready <= 1'b1;
89			end
90			
91			endmodule
92			

Branch Coverage:

Enabled Coverage	Active	Hits	Misses % Covered
-----	-----	----	-----

Branches	20	20	0	100.00
-----Branch Details-----				
Branch Coverage for file FIFO.v --				
-----IF Branch-----				
45		1032	Count coming in to IF	
45	1	288	if (FIFO_w && (FIFO_r FIFO_wready))	
		744	All False Count	
Branch totals: 2 hits of 2 branches = 100.00%				
-----IF Branch-----				
51		611	Count coming in to IF	
51	1	2	if (!RSTn)	
53	1	288	else if (FIFO_r && FIFO_rready)	
		321	All False Count	
Branch totals: 3 hits of 3 branches = 100.00%				
-----IF Branch-----				
54		288	Count coming in to IF	
54	1	18	if (rp == {BITNUM{1'b1}})	
56	1	270	else	
Branch totals: 2 hits of 2 branches = 100.00%				
-----IF Branch-----				
62		894	Count coming in to IF	
62	1	2	if (!RSTn)	
64	1	288	else if (FIFO_w && (FIFO_r FIFO_wready))	
		604	All False Count	
Branch totals: 3 hits of 3 branches = 100.00%				
-----IF Branch-----				
65		288	Count coming in to IF	
65	1	18	if (wp == {BITNUM{1'b1}})	
67	1	270	else	
Branch totals: 2 hits of 2 branches = 100.00%				
-----IF Branch-----				
73		894	Count coming in to IF	
73	1	2	if (!RSTn)	
75	1	14	else if (((rp == wp - 1'b1) (rp == {BITNUM{1'b1}}) && (wp == {BITNUM{1'b0}}))) && FIFO_r && (!FIFO_w))	
77	1	14	else if (!FIFO_rready && FIFO_w)	

864 All False Count

Branch totals: 4 hits of 4 branches = 100.00%

-----IF Branch-----

83		894	Count coming in to IF
83	1	2	if (!RSTn)
85	1	20	else if (((wp == rp - 1'b1) ((wp == {BITNUM{1'b1}}) && (rp == {BITNUM{1'b0}}))) && FIFO_w && (!FIFO_r))
87	1	20	else if (!FIFO_wready && FIFO_r && !FIFO_w)
		852	All False Count

Branch totals: 4 hits of 4 branches = 100.00%

Condition Coverage:

Enabled Coverage	Active	Covered	Misses	% Covered
-----	-----	-----	-----	-----
FEC Condition Terms	25	21	4	84.00

=====Condition Details=====

Condition Coverage for file FIFO.v --

-----Focused Condition View-----

Line 45 Item 1 (FIFO_w && (FIFO_r || FIFO_wready))

Condition totals: 3 of 3 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
FIFO_w	Y		
FIFO_r	Y		
FIFO_wready	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	FIFO_w_0	-
Row 2:	1	FIFO_w_1	(FIFO_r FIFO_wready)
Row 3:	1	FIFO_r_0	(FIFO_w && ~FIFO_wready)
Row 4:	1	FIFO_r_1	FIFO_w
Row 5:	1	FIFO_wready_0	(FIFO_w && ~FIFO_r)
Row 6:	1	FIFO_wready_1	(FIFO_w && ~FIFO_r)

-----Focused Condition View-----

Line 53 Item 1 (FIFO_r && FIFO_rready)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
FIFO_r	Y		
FIFO_rready	Y		
Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	FIFO_r_0	-
Row 2:	1	FIFO_r_1	FIFO_rready
Row 3:	1	FIFO_rready_0	FIFO_r
Row 4:	1	FIFO_rready_1	FIFO_r
-----Focused Condition View-----			
Line	54 Item	1	(rp == {4{{1}}})
Condition totals: 1 of 1 input term covered = 100.00%			
Input Term	Covered	Reason for no coverage	Hint
(rp == {4{{1}}})	Y		
Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(rp == {4{{1}}})_0	-
Row 2:	1	(rp == {4{{1}}})_1	-
-----Focused Condition View-----			
Line	64 Item	1	(FIFO_w && (FIFO_r FIFO_wready))
Condition totals: 3 of 3 input terms covered = 100.00%			
Input Term	Covered	Reason for no coverage	Hint
FIFO_w	Y		
FIFO_r	Y		
FIFO_wready	Y		
Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	FIFO_w_0	-
Row 2:	1	FIFO_w_1	(FIFO_r FIFO_wready)
Row 3:	1	FIFO_r_0	(FIFO_w && ~FIFO_wready)
Row 4:	1	FIFO_r_1	FIFO_w
Row 5:	1	FIFO_wready_0	(FIFO_w && ~FIFO_r)
Row 6:	1	FIFO_wready_1	(FIFO_w && ~FIFO_r)

-----Focused Condition View-----

Line 65 Item 1 (wp == {4{{1}}})

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(wp == {4{{1}}})	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-------	------	------------	--------------------------

Row 1:	1	(wp == {4{{1}}})_0	-
--------	---	--------------------	---

Row 2:	1	(wp == {4{{1}}})_1	-
--------	---	--------------------	---

-----Focused Condition View-----

Line 75 Item 1 (((rp == (wp - 1'b1)) || ((rp == {4{{1}}}) && (wp == {4{{0}}})) && FIFO_r) && ~FIFO_w)

Condition totals: 3 of 5 input terms covered = 60.00%

Input Term	Covered	Reason for no coverage	Hint
(rp == (wp - 1'b1))	Y		
(rp == {4{{1}}})	N	'_1' not hit	Hit '_1'
(wp == {4{{0}}})	N	'_1' not hit	Hit '_1'
FIFO_r	Y		
FIFO_w	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-------	------	------------	--------------------------

Row 1:	1	(rp == (wp - 1'b1))_0	~((rp == {4{{1}}}) && (wp == {4{{0}}}))
--------	---	-----------------------	---

Row 2:	1	(rp == (wp - 1'b1))_1	(~FIFO_w && FIFO_r)
--------	---	-----------------------	---------------------

Row 3:	1	(rp == {4{{1}}})_0	~(rp == (wp - 1'b1))
--------	---	--------------------	----------------------

Row 4:	***0***	(rp == {4{{1}}})_1	(~FIFO_w && FIFO_r && ~(rp == (wp - 1'b1)) && (wp == {4{{0}}}))
--------	---------	--------------------	---

Row 5:	1	(wp == {4{{0}}})_0	(~(rp == (wp - 1'b1)) && (rp == {4{{1}}}))
--------	---	--------------------	--

Row 6:	***0***	(wp == {4{{0}}})_1	(~FIFO_w && FIFO_r && ~(rp == (wp - 1'b1)) && (rp == {4{{1}}}))
--------	---------	--------------------	---

Row 7:	1	FIFO_r_0	((rp == (wp - 1'b1)) ((rp == {4{{1}}}) && (wp == {4{{0}}})))
--------	---	----------	---

Row 8:	1	FIFO_r_1	(~FIFO_w && ((rp == (wp - 1'b1)) ((rp == {4{{1}}}) && (wp == {4{{0}}}))))
--------	---	----------	--

Row 9:	1	FIFO_w_0	((rp == (wp - 1'b1)) ((rp == {4{{1}}}) && (wp == {4{{0}}})) && FIFO_r)
--------	---	----------	---

Row 10:	1	FIFO_w_1	((rp == (wp - 1'b1)) ((rp == {4{{1}}}) && (wp == {4{{0}}})) && FIFO_r)
---------	---	----------	---

-----Focused Condition View-----

Line 77 Item 1 (~FIFO_ready && FIFO_w)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

FIFO_rready	Y		
FIFO_w	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-------	------	------------	--------------------------

Row 1:	1	FIFO_rready_0	FIFO_w
Row 2:	1	FIFO_rready_1	-
Row 3:	1	FIFO_w_0	~FIFO_rready
Row 4:	1	FIFO_w_1	~FIFO_rready

-----Focused Condition View-----

Line 85 Item 1 (((wp == (rp - 1'b1)) || ((wp == {4{1}}) && (rp == {4{0}})))) && FIFO_w && ~FIFO_r

Condition totals: 3 of 5 input terms covered = 60.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

(wp == (rp - 1'b1))	Y		
(wp == {4{1}})	N	'_1' not hit	Hit '_1'
(rp == {4{0}})	N	'_1' not hit	Hit '_1'
FIFO_w	Y		
FIFO_r	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-------	------	------------	--------------------------

Row 1:	1	(wp == (rp - 1'b1))_0	~((wp == {4{1}}) && (rp == {4{0}}))
Row 2:	1	(wp == (rp - 1'b1))_1	(~FIFO_r && FIFO_w)
Row 3:	1	(wp == {4{1}})_0	~(wp == (rp - 1'b1))
Row 4:	***0***	(wp == {4{1}})_1	(~FIFO_r && FIFO_w && ~(wp == (rp - 1'b1)) && (rp == {4{0}}))
Row 5:	1	(rp == {4{0}})_0	(~(wp == (rp - 1'b1)) && (wp == {4{1}}))
Row 6:	***0***	(rp == {4{0}})_1	(~FIFO_r && FIFO_w && ~(wp == (rp - 1'b1)) && (wp == {4{1}}))
Row 7:	1	FIFO_w_0	((wp == (rp - 1'b1)) ((wp == {4{1}}) && (rp == {4{0}})))
Row 8:	1	FIFO_w_1	(~FIFO_r && ((wp == (rp - 1'b1)) ((wp == {4{1}}) && (rp == {4{0}}))))
Row 9:	1	FIFO_r_0	((wp == (rp - 1'b1)) ((wp == {4{1}}) && (rp == {4{0}}))) && FIFO_w
Row 10:	1	FIFO_r_1	((wp == (rp - 1'b1)) ((wp == {4{1}}) && (rp == {4{0}}))) && FIFO_w

-----Focused Condition View-----

Line 87 Item 1 ((~FIFO_wready && FIFO_r) && ~FIFO_w)

Condition totals: 3 of 3 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
FIFO_wready	Y		
FIFO_r	Y		
FIFO_w	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	FIFO_wready_0	(~FIFO_w && FIFO_r)
Row 2:	1	FIFO_wready_1	-
Row 3:	1	FIFO_r_0	~FIFO_wready
Row 4:	1	FIFO_r_1	(~FIFO_w && ~FIFO_wready)
Row 5:	1	FIFO_w_0	(~FIFO_wready && FIFO_r)
Row 6:	1	FIFO_w_1	(~FIFO_wready && FIFO_r)

Total Coverage By File (code coverage only, filtered view): 94.66%